

Code Submission: Edge-Based Semantic Voice Command Classifier

Candidate Name: [Your Name]

Problem Statement: Light Weight Speech Command Classifier for Edge Deployment

File Name: Your Name-Code.pdf

1. Project Directory Structure

```
.
├── requirements.txt
├── src/
│   ├── config.py
│   ├── dataset.py
│   ├── api.py
│   ├── utils.py
│   ├── dashboard/          # Dashboard Static Assets
│   │   ├── index.html
│   │   ├── style.css
│   │   └── app.js
│   ├── models/
│   │   ├── base.py
│   │   ├── tfidf_classifier.py
│   │   └── onnx_transformer.py
│   └── scripts/
│       ├── train_tfidf.py
│       ├── export_onnx.py
│       ├── benchmark.py
│       ├── interactive.py
│       └── dashboard.py    # Dashboard HTTP Server Backend
```

2. Requirements File (requirements.txt)

```
numpy>=1.22.0
scikit-learn>=1.0.0
sentence-transformers>=2.2.0
transformers>=4.30.0
tokenizers>=0.13.0
onnx>=1.14.0
onnxruntime>=1.15.0
optimum[onnxruntime]>=1.9.0
matplotlib>=3.5.0
seaborn>=0.11.0
pytest>=7.0.0
```

3. Configuration Module (src/config.py)

```
# src/config.py

from typing import Dict, List

CORE_COMMANDS: Dict[int, str] = {
    1: "activate do not disturb",
    2: "deactivate do not disturb",
    3: "decline the call",
    4: "pick up the call",
    5: "play the music",
    6: "pause the music",
    7: "play the next song",
    8: "play the previous song",
    9: "increase the volume",
    10: "decrease the volume"
}

EXTENSION_COMMANDS: Dict[int, str] = {
    11: "increase the brightness",
    12: "decrease the brightness",
    13: "start the vehicle",
    14: "stop the vehicle"
}

ALL_COMMANDS: Dict[int, str] = {**CORE_COMMANDS, **EXTENSION_COMMANDS}

COMMAND_TEMPLATES: Dict[int, List[str]] = {
    1: [
        "activate do not disturb", "turn on do not disturb", "enable do not disturb",
        "put the phone on do not disturb", "switch on do not disturb", "dnd mode turn on",
        "silence all notifications", "activate dnd"
    ],
    2: [
        "deactivate do not disturb", "turn off do not disturb", "disable do not disturb",
        "take off do not disturb", "switch off do not disturb", "turn off dnd",
        "deactivate dnd", "stop silencing notifications"
    ],
    3: [
        "decline the call", "decline call", "reject the call", "reject incoming call",
        "don't answer the call", "cancel incoming call", "ignore the call", "hang up the call"
    ]
}
```

```

    ],
    4: [
        "pick up the call", "pick up call", "answer the call", "accept the call",
        "answer incoming call", "receive call", "pick call", "connect the call"
    ],
    5: [
        "play the music", "play music", "start the music", "resume the music",
        "resume music playback", "start playing music", "turn on the music", "play song"
    ],
    6: [
        "pause the music", "pause music", "stop the music", "stop playing music",
        "pause playback", "halt the music", "suspend music", "pause song"
    ],
    7: [
        "play the next song", "play next song", "skip this song", "skip song",
        "next song", "go to next track", "next track please", "skip to next song"
    ],
    8: [
        "play the previous song", "play previous song", "go back a song", "previous song",
        "go to previous track", "replay the last song", "previous track please", "go back to previous track"
    ],
    9: [
        "increase the volume", "increase volume", "volume up", "make it louder",
        "raise the volume", "turn up the volume", "turn the volume up", "louder please"
    ],
    10: [
        "decrease the volume", "decrease volume", "volume down", "make it quieter",
        "lower the volume", "turn down the volume", "turn the volume down", "softer please"
    ],
    11: [
        "increase the brightness", "increase brightness", "brightness up", "make the screen brighter",
        "raise the brightness", "turn up brightness", "brighten the screen", "brighter please"
    ],
    12: [
        "decrease the brightness", "decrease brightness", "brightness down", "make the screen dimmer",
        "lower the brightness", "turn down brightness", "dim the screen", "dimmer please"
    ],
    13: [
        "start the vehicle", "start the engine", "turn on the engine", "start car",
        "ignition start", "turn on the car", "start vehicle engine", "switch on the engine"
    ],
    14: [
        "stop the vehicle", "stop the engine", "turn off the engine", "stop car",
        "kill the engine", "turn off the car", "shut down the engine", "switch off the car"
    ]
}

OOS_TEMPLATES: List[str] = [
    "what is the weather today", "how is the traffic on my route", "navigate to the nearest petrol station",
    "give me directions to the office", "what time is it", "set a timer for ten minutes",
    "call John", "send a text to mom", "search the web for electric vehicles", "tell me a joke",
    "hello", "hi assistant", "who are you", "open the window", "turn off the air conditioning",
    "turn on the ac", "adjust seat position", "show me the map", "open the sunroof", "check tire pressure"
]

COSINE_SIMILARITY_THRESHOLD = 0.65
TFIDF_CONFIDENCE_THRESHOLD = 0.45

```

4. Synthetic & Noisy Dataset Generator (src/dataset.py)

```

# src/dataset.py

import random
from typing import Dict, List, Tuple
from src.config import COMMAND_TEMPLATES, OOS_TEMPLATES, ALL_COMMANDS, CORE_COMMANDS

random.seed(42)

INDIAN_PHONETIC_REPLACEMENTS: Dict[str, List[str]] = {
    "volume": ["wolume", "wolyum", "volum"],
    "vehicle": ["vechicle", "vehical", "wehicle", "wehecle"],
    "brightness": ["brite-ness", "brightnes", "brightnessh"],
    "previous": ["preveous", "privius", "previos"],
    "music": ["musig", "museek", "mujic"],
    "decline": ["declinee", "dekline"],
    "disturb": ["desturb", "distarb"],
    "the": ["de", "ze", "d"]
}

INDIAN_PHRASING_PATTERNS: Dict[int, List[str]] = {
    5: ["music play", "song play please", "music start na"],
    6: ["music pause ya", "song stop please", "music pause na"],
    7: ["next track please", "next song play", "skip track na"],
    8: ["previous track please", "previous song play", "last song replay na"],
    9: ["volume increase please", "volume up ya", "volume increase na"],
    10: ["volume decrease please", "volume down ya", "volume decrease na"],
    11: ["brightness increase please", "brightness up ya", "brightness increase na"],
    12: ["brightness decrease please", "brightness down ya", "brightness decrease na"],
    13: ["car start please", "engine turn on na", "car start na"],
    14: ["car stop please", "engine turn off na", "car stop na"]
}

FILLERS: List[str] = ["um", "uh", "please", "can you", "hey", "like", "just", "please do one thing"]
INDIAN_POST_FILLERS: List[str] = ["na", "ya", "re", "please"]

```

```

def drop_articles(text: str) -> str:
    words = text.split()
    filtered_words = [w for w in words if w.lower() not in {"the", "a", "an"}]
    return " ".join(filtered_words)

def add_fillers(text: str) -> str:
    filler = random.choice(FILLERS)
    return f"{filler} {text}"

def apply_indian_accent_phonetics(text: str) -> str:
    words = text.split()
    augmented_words = []
    for w in words:
        w_lower = w.lower()
        if w_lower in INDIAN_PHONETIC_REPLACEMENTS:
            if random.random() < 0.7:
                augmented_words.append(random.choice(INDIAN_PHONETIC_REPLACEMENTS[w_lower]))
                continue
        if "th" in w_lower and random.random() < 0.5:
            w_lower = w_lower.replace("th", random.choice(["d", "z"]))
        if "w" in w_lower and random.random() < 0.4:
            w_lower = w_lower.replace("w", "v")
        elif "v" in w_lower and random.random() < 0.4:
            w_lower = w_lower.replace("v", "w")
        augmented_words.append(w_lower)
    return " ".join(augmented_words)

def apply_indian_syntax_and_fillers(cmd_id: int, text: str) -> str:
    if cmd_id in INDIAN_PHRASING_PATTERNS and random.random() < 0.4:
        return random.choice(INDIAN_PHRASING_PATTERNS[cmd_id])
    if random.random() < 0.5:
        post_filler = random.choice(INDIAN_POST_FILLERS)
        return f"{text} {post_filler}"
    return text

def apply_phonetic_typos(text: str) -> str:
    if len(text) < 5:
        return text
    chars = list(text)
    idx = random.randint(0, len(chars) - 1)
    if chars[idx].isalpha():
        if random.random() < 0.5:
            chars.insert(idx, chars[idx])
        else:
            chars[idx] = random.choice("abcdefghijklmnopqrstuvwxyz")
    return "".join(chars)

def generate_augmented_variants(cmd_id: int, template: str) -> List[str]:
    variants = []
    dropped = drop_articles(template)
    if dropped != template:
        variants.append(dropped)
    variants.append(add_fillers(template))
    if dropped != template:
        variants.append(add_fillers(dropped))
    variants.append(apply_indian_accent_phonetics(template))
    variants.append(apply_indian_accent_phonetics(dropped))
    variants.append(apply_indian_syntax_and_fillers(cmd_id, template))
    variants.append(apply_indian_syntax_and_fillers(cmd_id, dropped))
    variants.append(apply_phonetic_typos(template))
    return list(set([v.strip().lower() for v in variants if v.strip()]))

def generate_splits(include_extension: bool = True) -> Tuple[List[Dict[str, str]], List[Dict[str, str]], List[Dict[str, str]]:
    train_data: List[Dict[str, str]] = []
    clean_test: List[Dict[str, str]] = []
    noisy_test: List[Dict[str, str]] = []

    commands_to_use = ALL_COMMANDS if include_extension else CORE_COMMANDS

    for cmd_id, cmd_name in commands_to_use.items():
        templates = COMMAND_TEMPLATES[cmd_id]
        split_idx = max(1, int(len(templates) * 0.7))
        train_templates = templates[:split_idx]
        test_templates = templates[split_idx:]

        for t in train_templates:
            train_data.append({"text": t, "label": cmd_name})
            for variant in generate_augmented_variants(cmd_id, t):
                train_data.append({"text": variant, "label": cmd_name})

        for t in test_templates:
            clean_test.append({"text": t, "label": cmd_name})

        for t in test_templates:
            dropped = drop_articles(t)
            if dropped != t:
                noisy_test.append({"text": dropped, "label": cmd_name, "noise_type": "dropped_articles"})
            noisy_test.append({"text": add_fillers(t), "label": cmd_name, "noise_type": "filler_words"})
            noisy_test.append({"text": apply_indian_accent_phonetics(t), "label": cmd_name, "noise_type": "indian_phonetics"})
            noisy_test.append({"text": apply_indian_syntax_and_fillers(cmd_id, t), "label": cmd_name, "noise_type": "indian_syntax_and_fillers"})
            noisy_test.append({"text": apply_phonetic_typos(t), "label": cmd_name, "noise_type": "phonetic_typos"})

    random.shuffle(OOS_TEMPLATES)
    oos_split = int(len(OOS_TEMPLATES) * 0.7)
    for oos_text in OOS_TEMPLATES[:oos_split]:
        train_data.append({"text": oos_text, "label": "None"})

```

```

for oos_text in OOS_TEMPLATES[oos_split]:
    clean_test.append({"text": oos_text, "label": "None"})
    noisy_test.append({"text": oos_text, "label": "None", "noise_type": "out_of_scope"})

train_data = [dict(t) for t in {tuple(d.items()) for d in train_data}]
clean_test = [dict(t) for t in {tuple(d.items()) for d in clean_test}]

random.shuffle(train_data)
random.shuffle(clean_test)
random.shuffle(noisy_test)
return train_data, clean_test, noisy_test

```

5. Model Abstract Interface (src/models/base.py)

```

# src/models/base.py

from abc import ABC, abstractmethod
from typing import Tuple

class BaseCommandClassifier(ABC):
    @abstractmethod
    def train(self, train_data: list[dict[str, str]]) -> None:
        pass

    @abstractmethod
    def predict(self, text: str) -> Tuple[str, float]:
        pass

    @abstractmethod
    def save(self, path: str) -> None:
        pass

    @abstractmethod
    def load(self, path: str) -> None:
        pass

```

6. Baseline Classifier (src/models/tfidf_classifier.py)

```

# src/models/tfidf_classifier.py

import json
import math
import numpy as np
from typing import Tuple, Dict, Any, List
from src.models.base import BaseCommandClassifier
from src.config import TFIDF_CONFIDENCE_THRESHOLD

class TFIDFCommandClassifier(BaseCommandClassifier):
    def __init__(self, confidence_threshold: float = TFIDF_CONFIDENCE_THRESHOLD) -> None:
        self.confidence_threshold: float = confidence_threshold
        self.vocabulary: Dict[str, int] = {}
        self.idf: List[float] = []
        self.coef: List[List[float]] = []
        self.intercept: List[float] = []
        self.classes: List[str] = []
        self.sublinear_tf: bool = True

    def _tokenize(self, text: str) -> List[str]:
        clean_text = "".join([c.lower() if c.isalnum() or c.isspace() else " " for c in text])
        words = [w for w in clean_text.split() if w]
        tokens = list(words)
        for i in range(len(words) - 1):
            tokens.append(f"{words[i]} {words[i+1]}")
        return tokens

    def train(self, train_data: List[Dict[str, str]]) -> None:
        from sklearn.feature_extraction.text import TfidfVectorizer
        from sklearn.linear_model import LogisticRegression

        texts = [d["text"] for d in train_data]
        labels = [d["label"] for d in train_data]

        vectorizer = TfidfVectorizer(ngram_range=(1, 2), token_pattern=r"\b\w+\b", sublinear_tf=self.sublinear_tf, smooth_idf=True)
        X = vectorizer.fit_transform(texts)
        clf = LogisticRegression(max_iter=1000, C=2.0, class_weight='balanced')
        clf.fit(X, labels)

        self.vocabulary = vectorizer.vocabulary_
        self.idf = vectorizer.idf_.tolist()
        self.coef = clf.coef_.tolist()
        self.intercept = clf.intercept_.tolist()
        self.classes = clf.classes_.tolist()

    def _extract_tfidf_vector(self, text: str) -> np.ndarray:
        tokens = self._tokenize(text)
        vector = np.zeros(len(self.vocabulary))
        tf_counts: Dict[str, int] = {}
        for token in tokens:
            if token in self.vocabulary:
                tf_counts[token] = tf_counts.get(token, 0) + 1
        for token, count in tf_counts.items():
            idx = self.vocabulary[token]

```

```

        tf_val = 1.0 + math.log(count) if self.sublinear_tf else float(count)
        vector[idx] = tf_val * self.idf[idx]
    norm = np.linalg.norm(vector)
    if norm > 0:
        vector = vector / norm
    return vector

def predict(self, text: str) -> Tuple[str, float]:
    if not self.vocabulary:
        raise ValueError("Model has not been trained or loaded yet.")
    x = self._extract_tfidf_vector(text)
    coef_matrix = np.array(self.coef)
    intercepts = np.array(self.intercept)
    logits = np.dot(coef_matrix, x) + intercepts
    exp_logits = np.exp(logits - np.max(logits))
    probs = exp_logits / np.sum(exp_logits)
    best_idx = np.argmax(probs)
    pred_class = self.classes[best_idx]
    confidence = float(probs[best_idx])
    if confidence < self.confidence_threshold:
        return "None", confidence
    return pred_class, confidence

def save(self, path: str) -> None:
    model_data = {
        "vocabulary": self.vocabulary, "idf": self.idf, "coef": self.coef,
        "intercept": self.intercept, "classes": self.classes, "sublinear_tf": self.sublinear_tf,
        "confidence_threshold": self.confidence_threshold
    }
    with open(path, "w") as f:
        json.dump(model_data, f, indent=2)

def load(self, path: str) -> None:
    with open(path, "r") as f:
        model_data = json.load(f)
    self.vocabulary = model_data["vocabulary"]
    self.idf = model_data["idf"]
    self.coef = model_data["coef"]
    self.intercept = model_data["intercept"]
    self.classes = model_data["classes"]
    self.sublinear_tf = model_data.get("sublinear_tf", True)
    self.confidence_threshold = model_data.get("confidence_threshold", TFIDF_CONFIDENCE_THRESHOLD)

```

7. Quantized ONNX Transformer Classifier (src/models/onnx_transformer.py)

```

# src/models/onnx_transformer.py

import os
import json
import numpy as np
from typing import Tuple, Dict, List, Any
from src.models.base import BaseCommandClassifier
from src.config import COMMAND_TEMPLATES, COSINE_SIMILARITY_THRESHOLD

class ONNXTransformerClassifier(BaseCommandClassifier):
    def __init__(self, model_dir: str, similarity_threshold: float = COSINE_SIMILARITY_THRESHOLD, include_extension: bool =
        self.model_dir: str = model_dir
        self.similarity_threshold: float = similarity_threshold
        self.include_extension: bool = include_extension
        self.session = None
        self.tokenizer = None
        self.template_embeddings: Dict[int, np.ndarray] = {}
        self.commands: Dict[int, str] = {}

    def _mean_pooling(self, model_output: np.ndarray, attention_mask: np.ndarray) -> np.ndarray:
        token_embeddings = model_output
        input_mask_expanded = np.expand_dims(attention_mask, -1).astype(float)
        sum_embeddings = np.sum(token_embeddings * input_mask_expanded, axis=-1)
        sum_mask = np.clip(np.sum(input_mask_expanded, axis=-1), a_min=1e-9, a_max=None)
        mean_pooled = sum_embeddings / sum_mask
        norms = np.linalg.norm(mean_pooled, axis=-1, keepdims=True)
        norms = np.clip(norms, a_min=1e-9, a_max=None)
        return mean_pooled / norms

    def _embed(self, texts: List[str]) -> np.ndarray:
        if not self.session or not self.tokenizer:
            raise ValueError("Model is not loaded. Call load() first.")
        encoded = [self.tokenizer.encode(t) for t in texts]
        max_len = max(len(enc.ids) for enc in encoded)

        input_ids = []
        attention_mask = []
        for enc in encoded:
            ids = enc.ids + [0] * (max_len - len(enc.ids))
            mask = enc.attention_mask + [0] * (max_len - len(enc.attention_mask))
            input_ids.append(ids)
            attention_mask.append(mask)

        input_ids_np = np.array(input_ids, dtype=np.int64)
        attention_mask_np = np.array(attention_mask, dtype=np.int64)

        inputs = {"input_ids": input_ids_np, "attention_mask": attention_mask_np}
        model_inputs = [i.name for i in self.session.get_inputs()]
        if "token_type_ids" in model_inputs:

```

```

        inputs["token_type_ids"] = np.zeros_like(input_ids_np, dtype=np.int64)

        outputs = self.session.run(None, inputs)
        embeddings = self._mean_pooling(outputs[0], attention_mask_np)
        return embeddings

def train(self, train_data: List[Dict[str, str]]) -> None:
    from src.config import CORE_COMMANDS, ALL_COMMANDS
    self.commands = ALL_COMMANDS if self.include_extension else CORE_COMMANDS
    print("Pre-embedding command templates...")
    for cmd_id, cmd_name in self.commands.items():
        templates = COMMAND_TEMPLATES[cmd_id]
        embeddings = self._embed(templates)
        self.template_embeddings[cmd_id] = embeddings

def predict(self, text: str) -> Tuple[str, float]:
    if not self.template_embeddings:
        raise ValueError("Model has not been trained or template embeddings not cached.")
    input_emb = self._embed([text])
    best_cmd_id = -1
    best_score = -1.0
    for cmd_id, templates_emb in self.template_embeddings.items():
        similarities = np.dot(templates_emb, input_emb.T).squeeze(axis=1)
        max_sim = float(np.max(similarities))
        if max_sim > best_score:
            best_score = max_sim
            best_cmd_id = cmd_id
    if best_score < self.similarity_threshold:
        return "None", best_score
    pred_label = self.commands.get(best_cmd_id, "None")
    return pred_label, best_score

def save(self, path: str) -> None:
    serializable_embeddings = {str(cmd_id): emb.tolist() for cmd_id, emb in self.template_embeddings.items()}
    meta = {
        "include_extension": self.include_extension,
        "similarity_threshold": self.similarity_threshold,
        "embeddings": serializable_embeddings,
        "commands": {str(k): v for k, v in self.commands.items()}
    }
    with open(path, "w") as f:
        json.dump(meta, f, indent=2)

def load(self, path: str) -> None:
    import onnxruntime as ort
    model_path = os.path.join(self.model_dir, "model_quantized.onnx")
    if not os.path.exists(model_path):
        model_path = os.path.join(self.model_dir, "model.onnx")
    sess_options = ort.SessionOptions()
    sess_options.intra_op_num_threads = 1
    sess_options.inter_op_num_threads = 1
    self.session = ort.InferenceSession(model_path, sess_options, providers=["CPUExecutionProvider"])

    from tokenizers import Tokenizer
    self.tokenizer = Tokenizer.from_file(os.path.join(self.model_dir, "tokenizer.json"))

    if os.path.exists(path):
        with open(path, "r") as f:
            meta = json.load(f)
            self.include_extension = meta.get("include_extension", self.include_extension)
            self.similarity_threshold = meta.get("similarity_threshold", self.similarity_threshold)
            self.commands = {int(k): v for k, v in meta["commands"].items()}
            self.template_embeddings = {int(cmd_id): np.array(emb_list) for cmd_id, emb_list in meta["embeddings"].items()}
    else:
        self.train([])

```

8. Pipeline Interface (src/api.py)

```

# src/api.py

import os
from typing import Tuple, Dict, Any
from src.models.base import BaseCommandClassifier
from src.models.tfidf_classifier import TFIDFCommandClassifier
from src.models.onnx_transformer import ONNXTransformerClassifier

class VoiceCommandAPI:
    def __init__(self, model_type: str = "onnx") -> None:
        self.model_type: str = model_type.lower()
        self.classifier: BaseCommandClassifier
        project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

        if self.model_type == "tfidf":
            model_path = os.path.join(project_root, "models", "tfidf_model.json")
            self.classifier = TFIDFCommandClassifier()
            self.classifier.load(model_path)
        elif self.model_type == "onnx":
            model_dir = os.path.join(project_root, "models", "onnx_minilm")
            meta_path = os.path.join(model_dir, "onnx_metadata.json")
            self.classifier = ONNXTransformerClassifier(model_dir=model_dir)
            self.classifier.load(meta_path)
        else:
            raise ValueError("Invalid model_type. Must be 'tfidf' or 'onnx'.")

```

```

def _normalize(self, text: str) -> str:
    if not text:
        return ""
    return " ".join(text.lower().strip().split())

def classify(self, text: str) -> Dict[str, Any]:
    normalized_text = self._normalize(text)
    if not normalized_text:
        return {"input": text, "normalized": "", "prediction": "None", "score": 0.0, "status": "rejected"}
    pred_label, score = self.classifier.predict(normalized_text)
    return {
        "input": text, "normalized": normalized_text, "prediction": pred_label,
        "score": round(score, 4), "status": "accepted" if pred_label != "None" else "rejected"
    }

```

9. Evaluation Script (scripts/benchmark.py)

```

# scripts/benchmark.py

import os
import sys
import time
import numpy as np
from typing import List, Dict, Any, Tuple

sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from src.dataset import generate_splits
from src.api import VoiceCommandAPI
from src.utils import compute_metrics, compute_confusion_matrix, plot_confusion_matrix_text

def get_file_size_mb(path: str) -> float:
    if os.path.isdir(path):
        total_size = 0
        for dirpath, _, filenames in os.walk(path):
            for f in filenames:
                fp = os.path.join(dirpath, f)
                total_size += os.path.getsize(fp)
        return total_size / (1024 * 1024)
    elif os.path.exists(path):
        return os.path.getsize(path) / (1024 * 1024)
    return 0.0

def benchmark_model(api: VoiceCommandAPI, test_data: List[Dict[str, Any]], model_name: str) -> Dict[str, Any]:
    y_true = [d["label"] for d in test_data]
    y_pred = []
    latencies = []
    _ = api.classify("test query")
    for item in test_data:
        text = item["text"]
        start = time.perf_counter()
        res = api.classify(text)
        end = time.perf_counter()
        latencies.append((end - start) * 1000.0)
        y_pred.append(res["prediction"])
    metrics = compute_metrics(y_true, y_pred)
    return {"metrics": metrics, "latencies": latencies, "y_true": y_true, "y_pred": y_pred}

def run_benchmarks() -> None:
    _, test_clean, test_noisy = generate_splits(include_extension=True)
    project_root = os.path.dirname(os.path.abspath(__file__))
    tfidf_path = os.path.join(project_root, "models", "tfidf_model.json")
    onnx_dir = os.path.join(project_root, "models", "onnx_minilm")

    tfidf_api = VoiceCommandAPI(model_type="tfidf")
    onnx_api = VoiceCommandAPI(model_type="onnx")
    results = {}
    for name, api in [("TF-IDF Baseline", tfidf_api), ("Quantized ONNX MiniLM", onnx_api)]:
        clean_res = benchmark_model(api, test_clean, name)
        noisy_res = benchmark_model(api, test_noisy, name)
        model_size = get_file_size_mb(tfidf_path) if "TF-IDF" in name else get_file_size_mb(os.path.join(onnx_dir, "model_q"))
        results[name] = {"clean": clean_res, "noisy": noisy_res, "size_mb": model_size}

    for name, res in results.items():
        clean_metrics = res["clean"]["metrics"]
        noisy_metrics = res["noisy"]["metrics"]
        latencies = res["clean"]["latencies"]
        avg_lat, p95_lat = np.mean(latencies), np.percentile(latencies, 95)
        clean_y_true, clean_y_pred = res["clean"]["y_true"], res["clean"]["y_pred"]
        oos_true = sum(1 for gt, pd in zip(clean_y_true, clean_y_pred) if gt == "None" and pd == "None")
        oos_total = sum(1 for gt in clean_y_true if gt == "None")
        oos_rejection_rate = oos_true / oos_total if oos_total > 0 else 1.0
        valid_total = sum(1 for gt in clean_y_true if gt != "None")
        false_reject = sum(1 for gt, pd in zip(clean_y_true, clean_y_pred) if gt != "None" and pd == "None")
        false_rejection_rate = false_reject / valid_total if valid_total > 0 else 0.0

        print(f"{name} -> Size: {res['size_mb']:.2f}MB, Latency: {avg_lat:.2f}ms, Clean Acc: {clean_metrics['accuracy']:.2%}")

if __name__ == "__main__":
    run_benchmarks()

```

10. Dashboard Web Backend Server (scripts/dashboard.py)

```

# scripts/dashboard.py

import os
import sys
import json
import webbrowser
from http.server import HTTPServer, BaseHTTPRequestHandler
from typing import Dict, Any

# Ensure project root is in python path
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from src.api import VoiceCommandAPI

# Global APIs
APIs: Dict[str, VoiceCommandAPI] = {}

class DashboardHandler(BaseHTTPRequestHandler):
    def log_message(self, format: str, *args: Any) -> None:
        pass

    def do_GET(self) -> None:
        project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        dashboard_dir = os.path.join(project_root, "src", "dashboard")
        path = self.path
        if path == "/":
            path = "/index.html"
        file_path = os.path.join(dashboard_dir, path.lstrip("/"))

        if os.path.exists(file_path) and os.path.isfile(file_path):
            self.send_response(200)
            if file_path.endswith(".html"):
                self.send_header("Content-type", "text/html")
            elif file_path.endswith(".css"):
                self.send_header("Content-type", "text/css")
            elif file_path.endswith(".js"):
                self.send_header("Content-type", "application/javascript")
            self.end_headers()
            with open(file_path, "rb") as f:
                self.wfile.write(f.read())
        else:
            self.send_response(404)
            self.end_headers()

    def do_POST(self) -> None:
        if self.path == "/classify":
            content_length = int(self.headers["Content-Length"])
            post_data = self.rfile.read(content_length)
            try:
                data = json.loads(post_data.decode("utf-8"))
                text = data.get("text", "")
                model_type = data.get("model_type", "onnx")
                if model_type not in APIs:
                    APIs[model_type] = VoiceCommandAPI(model_type=model_type)
                result = APIs[model_type].classify(text)
                self.send_response(200)
                self.send_header("Content-type", "application/json")
                self.end_headers()
                self.wfile.write(json.dumps(result).encode("utf-8"))
            except Exception as e:
                self.send_response(500)
                self.end_headers()
                self.wfile.write(json.dumps({"error": str(e)}).encode("utf-8"))
        else:
            self.send_response(404)
            self.end_headers()

def run_server(port: int = 8000) -> None:
    project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    APIs["onnx"] = VoiceCommandAPI(model_type="onnx")
    server = HTTPServer(("localhost", port), DashboardHandler)
    url = f"http://localhost:{port}/"
    print(f"Server started at {url}")
    webbrowser.open(url)
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        server.server_close()

if __name__ == "__main__":
    run_server()

```

11. Live Evaluation: Cockpit Web Dashboard Link

An interactive cockpit web application is bundled to test and visualize this classification engine. The backend server is hosted directly by the Python script above.

Launch Instructions:

1. Open your terminal in the project directory, activate the environment, and run:

```
python scripts/dashboard.py
```


2. Navigate your web browser to this address:

□ <http://localhost:8000/>

3. Select your model backend, grant microphone access, speak commands, and watch the dashboard widgets react instantly.

```
<style type="text/css">@media print {
  *, :after, :before {background: 0 0 !important;color: #000 !important;box-shadow: none !important;text-shadow: none !im
  a, a:visited {text-decoration: underline}
  a[href]:after {content: " (" attr(href) ")"}
  abbr[title]:after {content: " (" attr(title) ")"}
  a[href^="#"]:after, a[href^="javascript:"]:after {content: ""}
  blockquote, pre {border: 1px solid #999;page-break-inside: avoid}
  thead {display: table-header-group}
  img, tr {page-break-inside: avoid}
  img {max-width: 100% !important}
  h2, h3, p {orphans: 3;widows: 3}
  h2, h3 {page-break-after: avoid}
}
html {font-size: 12px}
@media screen and (min-width: 32rem) and (max-width: 48rem) {
  html {font-size: 15px}
}
@media screen and (min-width: 48rem) {
  html {font-size: 16px}
}
body {line-height: 1.85}
.air-p, p {font-size: 1rem;margin-bottom: 1.3rem}
.air-h1, .air-h2, .air-h3, .air-h4, h1, h2, h3, h4 {margin: 1.414rem 0 .5rem;font-weight: inherit;line-height: 1.42}
.air-h1, h1 {margin-top: 0;font-size: 3.998rem}
.air-h2, h2 {font-size: 2.827rem}
.air-h3, h3 {font-size: 1.999rem}
.air-h4, h4 {font-size: 1.414rem}
.air-h5, h5 {font-size: 1.121rem}
.air-h6, h6 {font-size: .88rem}
.air-small, small {font-size: .707em}
canvas, iframe, img, select, svg, textarea, video {max-width: 100%}
body {color: #444;font-family: 'Open Sans', Helvetica, sans-serif;font-weight: 300;margin: 0;text-align: center}
img {border-radius: 50%;height: 200px;margin: 0 auto;width: 200px}
a, a:visited {color: #3498db}
a:active, a:focus, a:hover {color: #2980b9}
pre {background-color: #fafafa;padding: 1rem;text-align: left}
blockquote {margin: 0;border-left: 5px solid #7a7a7a;font-style: italic;padding: 1.33em;text-align: left}
li, ol, ul {text-align: left}
p {color: #777}</style>
```